RACS Research Project — Full Planning Document

Retrieval-Augmented Coverless Semantic Steganography

1. The Research Goal (Restated Clearly)

Hypothesis we're testing:

A retrieval-based reference image (found via semantic search in an existing image database, then lightly edited) can replace a fully AI-generated reference image in coverless steganography — achieving comparable recovery quality and security to the state-of-the-art, while being significantly faster and cheaper to compute.

Why this matters: The current best system (AgentSemSteCom) solved a real security flaw (guessable private keys) but left an unsolved efficiency problem — it generates a brand-new image from scratch, every single time, using a heavy diffusion model. This is impractical for real-time or resource-constrained 6G edge devices, which is the exact use case the field is aiming for.

Important self-check: Simply "swapping generation for retrieval" is a good starting point, but by itself it risks being seen as an engineering optimization rather than a research contribution. Section 4 of this document lays out how to make this a genuinely deeper, structurally novel piece of research — this is the most important section to read carefully.

2. What Needs to Be Built (System Components)

Break the system into independent modules — build and test each one separately before combining.

2.1 Retrieval Database

- A subset of the LAION dataset (start with ~50k–100k images; scale up later if needed)
- CLIP embeddings computed for every image in the subset
- A FAISS (or similar) vector index for fast nearest-neighbor search
- **Deliverable:** given a text description, return the top-K most similar images in milliseconds

2.2 Semantic Extraction Module

- An LLM (or vision-language model like BLIP) that looks at the secret image and produces a public text description
- A structure/implicit-feature extractor (segmentation map for general objects, pose/skeleton map for faces)
- **Deliverable:** secret image → (public description, structure map)

2.3 Reference Image Acquisition Module (our core change)

- Text description → CLIP embedding → FAISS search → top-K candidate images
- Digital token → deterministically selects one candidate from the top-K set
- Light editing step (e.g., SDEdit-style partial diffusion, or a lightweight image-to-image adjustment) to align the selected image's structure with the secret image's structure map
- **Deliverable:** secret image + token → reference image (fast path, no full generation)

2.4 Hiding/Recovery Core (Coverless Steganography)

- An invertible mechanism to hide the secret image inside the reference image, producing a stego image
- Must be exactly reversible (or near-exactly) using the same token + reference image
- Options: reimplement EDICT-style coupled diffusion (like the base paper), or explore alternatives (see Section 4)
- **Deliverable:** (secret image, reference image, token) → stego image → (reversible) → recovered secret image

2.5 Channel Simulation (Lightweight, Backdrop Only)

- Simple AWGN (Additive White Gaussian Noise) simulation at various SNR levels (5–20 dB)
- No need for deep wireless/PHY-layer modeling — this is context, not our research focus
- **Deliverable:** stego image → noisy channel → reconstructed stego image

2.6 Evaluation & Security Testing Suite

- Quality metrics: PSNR, SSIM, MSE, LPIPS
- Speed/efficiency metrics: inference time, GPU memory, FLOPs estimate
- Security tests: steganalysis detectors (XuNet, optionally SRNet), legitimate-vs-eavesdropper comparison, wrong-token/no-token failure tests
- **Deliverable:** a repeatable benchmark script producing all paper figures/tables

3. What You Need to Learn (Skills & Knowledge Roadmap)

This is organized by priority — top items are foundational and needed almost immediately; lower items can be learned just-in-time as you reach that part of the project.

Tier 1 — Needed in Week 1–2 (Core Foundations)

Topic	Why	What to learn
Diffusion models basics	Everything in this project builds on diffusion (DDPM/DDIM)	How forward noising + reverse denoising works, what a "latent" is, what a "sampling step" means
CLIP embeddings	Powers our entire retrieval system	How CLIP maps images and text into the same embedding space; cosine similarity search
Vector search / FAISS	How retrieval actually works at scale	Approximate nearest neighbor search, index types (flat vs. IVF vs. HNSW), trade-offs
PyTorch + Hugging Face `diffusers`	Your main coding toolkit	Loading pretrained models, running inference pipelines, basic fine-tuning

Tier 2 — Needed by Week 3–5

Topic	Why	What to learn
ControlNet	Used to guide generation/editing with structural maps	How structural conditioning is injected into diffusion models
Invertible/reversible diffusion (EDICT concept)	Core to how hiding/recovery works	Why standard DDIM isn't perfectly reversible, how coupled transformations fix this
Image-to-image editing (SDEdit-style)	Needed for the "light editing" step	How partial diffusion (starting from a real image instead of noise) works

Topic	Why	What to learn
Evaluation metrics: PSNR, SSIM, LPIPS	You'll compute these constantly	What each metric actually measures and why we use multiple (pixel-level vs. structural vs. perceptual)

Tier 3 — Needed by Week 6–9

Topic	Why	What to learn
Steganalysis basics	For security evaluation	How detectors like XuNet/SRNet work (CNN-based classifiers trained to spot hidden data)
Information theory basics (entropy, search space)	For formalizing security claims	Concepts like "bits of security," brute-force search space size — needed if you pursue the theoretical security angle (Section 4)
Basic wireless communication concepts	Just enough to simulate the channel correctly	AWGN, SNR, what "channel capacity" roughly means (no need for deep PHY-layer expertise)

Tier 4 — Ongoing / As Needed

Topic	Why
Reading and dissecting research papers efficiently	You'll need to read 15-20+ related papers over the project
LaTeX (for the final paper)	Standard for research paper formatting
Experiment tracking (e.g., Weights & Biases, or simple structured logging)	You'll run many parameter sweeps; need organized results
Git/version control discipline	Essential once multiple people touch the same codebase

This is the most important section. Simply retrieving instead of generating is a good idea, but to make this genuinely strong research (not just an engineering swap), we should pick at least 1–2 of the following **structural/architectural contributions** to build in — these change *how the system reasons*, not just *which model it calls*.

4.1 Learned Retrieval Embedding for "Steganographic Suitability" (Recommended — High Impact)

The problem with plain CLIP retrieval: CLIP embeddings are trained for general visual-semantic similarity — they have no notion of which images are *good containers* for hiding secret data (e.g., images with enough texture/entropy to mask perturbations, or ones that are more resistant to steganalysis).

The idea: Instead of using off-the-shelf CLIP similarity, **fine-tune or learn a secondary scoring function** that reranks retrieved candidates based on *steganographic suitability* — not just "does this look like the description" but "will this image hide data well and resist detection."

Why this is structural, not cosmetic: This turns retrieval itself into a novel object of study — a retrieval system *designed for security*, not just repurposed from image search. This is a genuinely new idea nobody has published yet in this space.

4.2 Agentic Retrieve-or-Generate Hybrid Decision Layer (Recommended — Strong Novelty + Practical)

The problem: Pure retrieval will sometimes fail — if the secret image is unusual and no good match exists in the database, forcing a retrieval-only approach will hurt quality.

The idea: Add a genuine **decision-making agent layer**: after retrieval, the system evaluates whether the best candidate is "good enough" (via a similarity/quality threshold). If yes → use retrieval (fast path). If no → fall back to full generation (slow but higher quality, guided by the failed retrieval attempt for context).

Why this is structural: This isn't swapping one component for another — it's introducing a **new architectural layer** (an agentic decision/routing mechanism) that doesn't exist in either the base paper or standard RAG systems. It also directly ties into the "agentic AI" framing that the whole 6G/intellicise research area is built around — you're extending their *agentic* philosophy in a genuinely new direction (adaptive resource-aware decision making), not undermining it.

4.3 New Security Formalization: From "Guess the Seed" to "Guess the Index" (Recommended — Theoretical Contribution)

The problem: The base paper's security argument is empirical (they measure PSNR gaps) but not formally modeled. Since our system's token controls a **discrete selection** (which of K retrieved images), rather than continuous noise, we can formalize security differently — and more rigorously.

The idea: Derive a simple probabilistic/information-theoretic bound: if an eavesdropper doesn't know the token, their probability of picking the correct reference image is roughly 1/K (where K is the retrieval pool size), giving a clean, provable "bits of security" measure (e.g., log2(K) bits). This is something the base paper's noise-seed approach can't cleanly express, since continuous noise spaces don't have as neat a "guessing probability" framing.

Why this is structural: This adds a **theoretical security guarantee** section to your paper (not just empirical testing) — a meaningfully different and stronger contribution than the base paper offers.

4.4 Structure-Aware Multi-Modal Retrieval (Optional — Medium Effort, Medium Novelty)

The problem: Text-only retrieval may return images with the right "content" but wrong pose/layout, requiring heavier editing to fix.

The idea: Build a joint retrieval score combining (a) CLIP text-similarity and (b) a structural similarity score (comparing segmentation/pose maps) — retrieving images that match both *what* the image shows and *how* it's arranged.

Why this could matter: This is a genuine architectural change to the retrieval system (multi-modal, not single-modal), but is more effort to implement well (needs a compatible structural embedding space) — consider this a stretch goal if time allows after 4.1/4.2 are done.

4.5 Token-Conditioned Retrieval Reranking (Optional — Advanced, High Novelty, Higher Risk)

The problem: In our simple design, the token just *picks an index* from an already-computed retrieval list — the token doesn't influence the search itself.

The idea: Design a mechanism where the digital token perturbs the query embedding itself before search (e.g., adding a token-derived offset vector to the CLIP embedding), so different tokens don't just pick different items from the *same* list — they generate genuinely different *search results*. This deepens the security integration into the retrieval math itself.

Why this is risky: This requires careful design to avoid breaking retrieval quality (a bad offset could return irrelevant images) — good as an ambitious stretch goal, not a core deliverable, given the 3-month timeline.

Recommended Priority for a 3-Month Project

1. **Core RACS pipeline** (retrieval + light edit + hiding) — foundational, must-have
2. **4.2 — Agentic retrieve-or-generate hybrid** — high value, moderate effort, strengthens the "agentic AI" narrative
3. **4.3 — Security formalization** — low implementation cost (mostly math/analysis), high payoff for paper rigor
4. **4.1 — Learned steganographic-suitability reranking** — high value but higher effort (needs training data/labels for "good hiding containers") — attempt if time allows after 1–3 are solid
5. **4.4 / 4.5 — stretch goals** — only if ahead of schedule

5. Evaluation Plan (What Experiments to Run)

Experiment	Purpose
RACS vs. AgentSemSteCom (full pipeline comparison)	Main result — quality vs. speed trade-off
Retrieval-only vs. retrieval + light editing	Justifies the editing step's necessity
Varying top-K retrieval pool size (e.g., 10 / 100 / 1,000 / 10,000)	Security/quality trade-off curve; also feeds into 4.3's theoretical bound
Legitimate receiver vs. wrong-token vs. no-token eavesdropper	Core security test, same structure as base paper
Different SNR levels (5–20 dB)	Robustness under realistic wireless noise
(If pursuing 4.2) Hybrid vs. pure-retrieval vs. pure-generation	Shows the value of the adaptive decision layer
Steganalysis detection accuracy (XuNet, + SRNet if time allows)	Covertness evaluation
Inference time & GPU memory benchmarks	Headline efficiency result

6. Realistic Timeline (13 Weeks)

Weeks	Focus
1–2	Learn Tier 1 topics; set up environment; build small-scale retrieval database + FAISS search
3–4	Build semantic extraction module; implement basic hiding/recovery core (test on generated cover images first)
5–6	Combine retrieval + light editing into the reference acquisition module; first end-to-end RACS pipeline test
7–8	Implement chosen structural contribution(s) from Section 4 (recommend starting with 4.2 and 4.3)
9–10	Full evaluation sweep — quality, speed, security metrics across all experiments in Section 5
11	Ablation studies, refine weak results, re-run anything inconclusive
12–13	Write paper draft, prepare figures/tables, internal review, revise

7. Open Questions to Resolve Early (Discuss With Mentor)

1. Which structural contribution(s) from Section 4 should we commit to, given our timeline and team size?
2. What's our target LAION subset size — balance between "enough diversity for good retrieval" and "manageable to embed/index quickly"?
3. Should we implement EDICT ourselves, or explore a simpler/different invertible hiding mechanism (this could itself be a small structural improvement, e.g., using a genuinely invertible architecture like a normalizing flow instead of EDICT's workaround)?
4. What's our fallback plan if pure retrieval quality is too weak even after editing — do we commit to the hybrid agentic layer (4.2) as a safety net from the start, rather than as an optional add-on?
5. What venue are we targeting for submission, and what's the required format/deadline — this affects how much time to allocate to writing vs. experiments.

8. One-Paragraph Summary (For Quick Reference)

We are building RACS, a system that replaces the expensive "generate a fake reference image from scratch" step in coverless semantic steganography with "retrieve a similar real image from a large database, then lightly edit it." Beyond this core swap, we are adding genuine architectural novelty: an agentic decision layer that intelligently chooses between fast retrieval and slower generation depending on match quality, and a formal security analysis that reframes the digital token's role as controlling a discrete selection (with a provable "bits of security" guarantee) rather than continuous random noise. To execute this, we need to learn diffusion models, CLIP embeddings, vector search, ControlNet-based conditioning, and steganalysis fundamentals — most of which can be picked up in the first few weeks. Our evaluation will directly compare against the base paper (AgentSemSteCom) on quality, security, and — most importantly — speed and computational cost, which the base paper never measured and explicitly left as open future work.